The qLDPC Challenge

A public, automatically verified leaderboard for quantum low-density parity-check codes

Unitary Foundation `github.com/unitaryfoundation/qldpc-challenge`

June 18, 2026

Abstract

The qLDPC Challenge is a public leaderboard for quantum low-density parity-check codes. A participant submits a single JSON file describing one CSS code; a verifier recomputes the code's properties and, if it holds up, the code is merged and appears on the board. Three principles shape the design. (i) *Everything cheap is trustless and mandatory*: the verifier recomputes n, k, CSS commutation, check weight, and geometric locality from scratch over $\mathbb{F}_2$ — a wrong claimed k or a non-commuting "stabilizer" is rejected in milliseconds. (ii) *Distance is split into a self-certifying upper bound and a separately earned exact tier*: because computing distance is NP-hard, a submitter attaches an explicit logical-operator *witness* that proves $d \leq$ value with no trust required; upgrading a claim to *exact* requires a separate, more expensive server certification. (iii) *A code is not a single number*: there is no one winner but a set of *tracks* (hard-constraint categories), and within each track the ranking is a Pareto frontier over (n, k, d) rather than a scalar. The whole pipeline — validator, distance certifier, scorer, static-site generator — lives in the repository and runs in CI on every pull request.

1 Overview

A submission is one JSON file placed under `codes/`, conforming to `schema/code.schema.json` (described in `schema/SCHEMA.md`). A contributor opens a pull request adding the file; GitHub Actions runs the verifier (`verify/qldpc_verify.py`), and a green check means the code's cheap, trustless properties are confirmed. After merge, `site/build.py` regenerates the static leaderboard into `docs/` (an index with per-track Pareto plots and tables, a page per code, a reference list, and status badges). The boards are seeded with the planar codes of Liang, Eberhardt, and Chen (arXiv:2504.08887) and other literature baselines, so a new entry is always measured against the published state of the art rather than an empty board.

Two things are deliberately *not* asked of the submitter: a proof that their distance is exact (that is the server's job), and a single headline rank (the board is a Pareto frontier). Everything else — the parity checks, the claimed n, k, d, an optional layout, and provenance — is supplied and then recomputed or witnessed by the verifier.

2 Why a leaderboard of tracks, not one number

A quantum code trades several quantities against one another — physical qubits n, logical qubits k, distance d, maximum check weight, and geometric locality — and separately has a decoding performance. Collapsing all of that into one scalar would reward gaming one axis at the expense of the rest. So the board is stratified:

Tracks are hard-constraint categories. A code only appears on a track's board if it satisfies that track's constraints, which the verifier checks. The tracks live in `TRACKS.md`; the initial set is:

- `weight-6` — CSS codes with all checks of weight ≤ 6, any connectivity (the headline frontier), with `weight-4` and `weight-8` as sub-/super-threshold siblings;
- `2d-local-bilayer` — geometrically 2D-local on up to two physical layers (the flip-chip regime), requiring a `locality` block with `layers` ≤ 2, ranked within a maximum interaction radius;
- `2d-local-single` — 2D-local on a single layer (surface-code-like connectivity); stricter, mostly a baseline track;
- `bivariate bicycle (periodic)` — bivariate bicycle codes on a torus, no 2D-local layout, seeded with the canonical codes of Bravyi et al. (arXiv:2308.07915);
- `generalized bicycle` — univariate quasi-cyclic codes from two circulant polynomials, seeded from the literature (arXiv:2203.17216);
- `topological` — foundational surface/toric/color codes, exact by construction;
- `decoding` (planned) — a server-run logical-error-rate / threshold track under a fixed circuit-level noise model, which needs decoder sandboxing and lands after the parameter tracks.

A code may enter several tracks at once (it lists them all in `tracks`). New tracks can be proposed by PR.

Pareto frontier, not a single winner. Within a track, a code is on the frontier if no other code in that track dominates it — i.e. no other has $n' \leq n$, $k' \geq k$, $d' \geq d$ with at least one strict inequality. There can be many co-leaders. The board also offers a *cell view*: a grid keyed by (k, d), each cell holding the smallest known n, which is the code-tables-style view people usually screenshot.

kd^2/n as a sortable column, not the rank. The encoding-efficiency figure kd^2/n that the 2D-locality literature uses is reported and sortable, and serves as a reasonable per-track headline, but it never collapses the frontier into one number.

3 Background: the kd^2/n metric and the BPT bound

For an $[[n, k, d]]$ stabilizer code, the Bravyi–Poulin–Terhal (BPT) bound states that any geometrically 2D-local code obeys

$$\frac{kd^2}{n} \;\leq\; c(r, w, q), \tag{1}$$

where the constant c depends only on the interaction range r, the maximum stabilizer weight w, and the on-site dimension q — not on n. The optimal constant is open, which makes "within a fixed locality model, maximize kd^2/n" a well-posed target for the 2D-local tracks. The bound is asymptotic, so small finite codes routinely exceed any limiting family's constant; this is exactly why the board buckets and compares like with like rather than ranking a $[[120, \dots]]$ entry against a $[[10^4, \dots]]$ one. Reference points the boards are seeded with (all $q = 2$; w = max check weight):

Code	$[[n, k, d]]$	topology	w	kd^2/n
Toric (Kitaev), $L \times L$	$[[2L^2, 2, L]]$	periodic	4	1
Rotated planar surface	$[[d^2, 1, d]]$	**planar**	4	1
Gross code (IBM)	$[[144, 12, 12]]$	periodic	6	12
Liang–Eberhardt–Chen	$[[450, 11, 15]]$	**planar**	6	5.50
Liang–Eberhardt–Chen	$[[282, 12, 14]]$	**planar**	8	8.34

The planar weight-6/8 state of the art is set by Liang, Eberhardt, and Chen (arXiv:2504.08887), who convert bivariate-bicycle codes to open boundaries via boundary anyon condensation plus a "lattice grafting" optimization, with all reported distances computed exactly by integer programming — exactly the referee posture this challenge adopts. Whether the gap to the periodic weight-6 records is intrinsic to open boundaries or an artifact of a restricted search is the kind of question a live leaderboard can resolve.

4 Submission format

A submission is one JSON object (`schema_version "0.1"`, `code_type "CSS"` — the only type in v0.1) with:

- `n` — physical qubit count; must match the indices used in `checks`.
- `k` — *claimed* logical qubit count; the verifier recomputes it and requires an exact match.
- `checks.X`, `checks.Z` — the parity checks as sparse supports: each is a list of checks, each check a sorted list of distinct 0-based qubit indices. Thus H_X has `len(checks.X)` rows.
- `distance.d` — claimed distance $= \min(d_X, d_Z)$, and per side `distance.X`/`distance.Z`, each a $\{$`value`, `confidence`, `witness`$\}$ triple, where `confidence` is `"upper_bound"` or `"exact"` and `witness` is the support of a logical operator of that Pauli type and weight `value`.
- `locality` (optional; required for the 2D-local tracks) — `coordinates` (one $[x, y]$ per qubit), `layers`, and a claimed `interaction_radius` (recomputed by the verifier).
- `provenance` — `authors`, `construction`, optional `references` (arXiv id / DOI resolved against `refs.bib`), `date`, `notes`.
- `tracks` — the track ids this code is entered into.

Verify locally before opening the PR (the project uses `uv`):

```
uv run python verify/qldpc_verify.py codes/your-code.json
```

The verifier prints a JSON report — per-check pass/fail, the computed n, k, ranks, max_check_weight, interaction_rad and an `earned_distance` block giving the tier each side actually earned — and exits 0 iff every required check passes.

5 What the verifier checks

A submission is rejected unless all of the following hold. Checks (V1)–(V7) are polynomial-time and run in CI on every PR; the exact distance tier (Section 6) is a separate server step.

(V1) **Schema and indices.** The document conforms to `code.schema.json`, and every qubit index in `checks` is in $[0, n)$.

(V2) **No collapsed checks.** No qubit index is repeated within a single check (a repeat would XOR away and silently change the code).

(V3) **CSS commutation.** $H_X H_Z^\top = 0$ over $\mathbb{F}_2$.

(V4) **Logical dimension.** The verifier recomputes $k = n - \mathrm{rank}_{\mathbb{F}_2}(H_X) - \mathrm{rank}_{\mathbb{F}_2}(H_Z)$ and requires it to equal the claimed k.

(V5) **Distance witnesses.** For each provided side, the witness v must (a) have weight equal to the claimed `value`, (b) lie in the kernel of the opposite checks (an X-witness commutes with H_Z), and (c) be nontrivial, i.e. not in the rowspace of its own checks (not a stabilizer product). This certifies $d_{\text{side}} \leq$ `value` as an *upper bound* with no trust required.

(V6) **Distance consistency.** The claimed d equals the minimum over the witnessed sides.

(V7) **Locality (if present).** `coordinates` cover all n qubits, and the recomputed maximum check diameter is $\leq$ the claimed `interaction_radius`.

Every `exact` confidence claim is *downgraded to upper_bound here* and flagged for server certification — the cheap PR check never upgrades a distance to exact.

6 Computing and certifying the distance

The distance is the minimum weight over nontrivial logicals; for a CSS code $d = \min(d_X, d_Z)$. The problem is NP-hard in general, and the hardness is asymmetric: exhibiting a low-weight logical gives only an *upper* bound, while certifying $d \geq D$ is the hard direction, and a submitter has every incentive to overclaim. The challenge handles this with two tiers.

Upper-bound tier (trustless, in CI). The submitter's witness is checked exactly as in (V5). A valid witness proves $d_{\text{side}} \leq$ `value`; the board labels such a code $d \leq$ and lets anyone climb instantly, since the arithmetic is checked rather than trusted.

Exact tier (server-certified). To upgrade a side to $d =$, a maintainer runs `verify/certify.py`, which proves no lighter nontrivial logical exists via a bounded integer program (scipy/HiGHS, no commercial solver). For each logical generator t_L of the relevant type it asks whether a v exists with

$$Hv \equiv 0, \qquad \langle v, t_L \rangle \equiv 1 \pmod 2, \qquad \mathrm{wt}(v) \leq \texttt{value} - 1, \tag{2}$$

the mod-2 constraints linearized by integer slacks. If every generator on both sides is proven *infeasible*, d is exact; if a lighter logical is found, the claim is *refuted*; if the solver hits its time limit, the side honestly remains an upper bound. A successful run writes a certificate to `certs/<slug>.json` (with `d_exact`, per-side notes, solver, and per-solve time limit), which is what upgrades a code's board label to $d =$. Exact records outrank equal upper-bound ones; nothing is ever silently upgraded.

7 Duplicate detection

To keep the board honest about novelty, the verifier fingerprints every code two ways. The reduced-row-echelon forms of H_X, H_Z pin the exact stabilizer group: two codes with the same RREF fingerprint are *identical* (same labeling) and a duplicate is a hard CI error. A finer, permutation-invariant Weisfeiler–Leman color refinement on the Tanner graph gives a signature that any two qubit-permuted copies must share; a collision *flags* a possible equivalent for human review (WL is a strong necessary condition, not a complete equivalence test).

8 Repository layout and automation

Path	Contents
`schema/`	submission format (JSON Schema + `SCHEMA.md`)
`verify/`	verifier: `gf2.py` (GF(2) core), `qldpc_verify.py` (checker), `certify.py`/`certify_all.py` (exact tier), `verify_all.py`, tests
`examples/`	worked examples that pass verification
`codes/`	accepted submissions (the leaderboard data)
`certs/`	server-side exact-distance certificates
`site/`	`build.py`, which generates the static site
`docs/`	the generated site (index, per-code pages, references, badges)
`refs.bib`	bibliography; `references.html` is generated from it
`TRACKS.md`	the tracks and how ranking works

CI (`.github/workflows/verify.yml`) runs the verifier self-tests (`test_verifier.py`, which must reject malformed and dishonest submissions) and then `verify_all.py` over every file in `codes/` and `examples/`, failing if any code is invalid or if two entries share an identical fingerprint. The badges in the README and the figures in `docs/stats.json` are regenerated on every build from the live board, so they track the current numbers rather than a hand-typed count. At the time of writing the board holds on the order of three dozen verified codes (around twenty certified exact) across six populated tracks, with a best $kd^2/n \approx 14.2$ (a $[[126, 28, 8]]$ weight-8 entry); the live figures are in `docs/stats.json`.

9 Open questions this could surface

- Can planar weight-6 codes close the gap to the periodic records, or is part of it intrinsic to open boundaries?
- Do bulk stabilizers or boundary geometries outside the families searched so far beat the Liang et al. records?
- The small-lattice distances of the seed codes are set by Sierpinski-type *fractal* logical operators rather than strings — can this regime be engineered deliberately for better finite-size codes?

Key references

- Liang, Eberhardt, Chen, arXiv:2504.08887 — planar open-boundary qLDPC codes: the seed planar baseline (anyon-condensation + lattice-grafting construction).
- Bravyi, Cross, Gambetta, Maslov, Rall, Yoder, Nature **627**, 778 (2024), arXiv:2308.07915 — the $[[144, 12, 12]]$ gross code and bivariate-bicycle codes (periodic-track seed).
- Wang, Pryadko, Symmetry **14**, 1348 (2022), arXiv:2203.17216 — distance bounds for generalized bicycle codes (generalized-bicycle-track seed).
- Kitaev, Ann. Phys. **303**, 2 (2003), arXiv:quant-ph/9707021 — the toric code; Steane, arXiv:quant-ph/9601029 — CSS codes (topological-track baseline).
- Panteleev, Kalachev, STOC 2022, arXiv:2111.03654, and Tillich, Zémor, IEEE Trans. IT **60**, 1193 (2014), arXiv:0903.0566 — good and hypergraph-product qLDPC codes.
- Pryadko, Shabashov, Kozin, `QDistRnd`, JOSS **7**, 4120 (2022) — randomized distance estimation, usable as a refutation oracle; Perlin, `qLDPC` (https://github.com/qLDPCOrg/qLDPC) — a convenient way to construct codes and export their parity checks.